```
class Stack {
push(x)
pop()
}
```

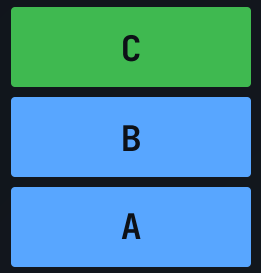
TEMA 2

Piles i Cues

Estructures de dades lineals amb accés restringit

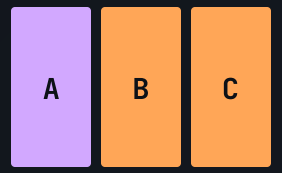
Dos contenidors fonamentals que manipulen elements amb estratègies d'accés oposades: LIFO (Last In First Out) per a piles i FIFO (First In First Out) per a cues.

PILA (LIFO)



↓ push ↑ pop

CUA (FIFO)



← dequeue | enqueue →

⚡ Accés O(1) 🔄 Operacions simples 🎯 Aplicacions reals

```
class Queue {
enqueue(x)
dequeue()
}
```

TEMA 2 - CONCEPTES C++

Característiques avançades del llenguatge

Convencions, optimitzacions i mecanismes de validació

Convencions de noms

Els membres privats es diferencien amb sufixos o prefixos per millorar la llegibilitat del codi.

```
class Data {
  int dia_; // Suffix
  int _mes; // Prefix
  int m_any; // m_prefix
};
```

Mètodes inline

Implementació dins la classe. Avantatge: eficiència. Desavantatge: pot augmentar la mida del codi compilat.

```
class Comptador {
  int c_;
public:
  void incrementa() {
    c_++; // Inline!
  }
};
```

Listes d'inicialitzadors

Mecanisme especial per inicialitzar membres abans del cos del constructor. Sintaxi: : membre_(valor)

```
Gran::Gran(int a)
  : petit_(a) {
  // Cos del constructor
}
```

assert per precondicions

Converteix pre-condicions en cridades assert. Atura el programa si la condició és falsa.

```
int busca(int des) {
  assert(des ≥ 0);
  // ... implementació
}
```

TEMA 2 - PILES

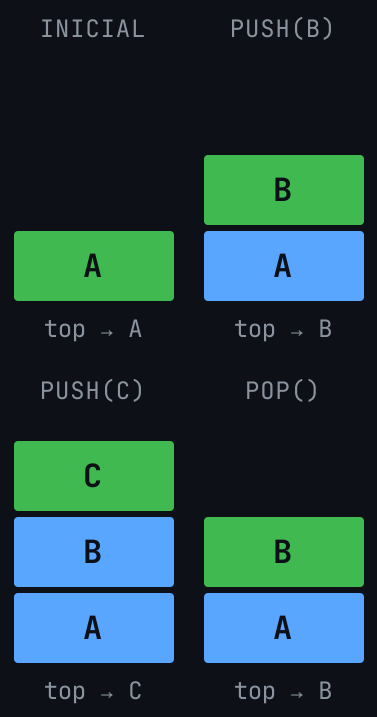
Piles: Definició i Operacions

TAD LIFO (Last In First Out) - L'últim que entra és el primer que surt

> Una **pila** és un contenidor molt simple on només podem manipular l'element d'un extrem (el **tope**. Els elements s'afegeixen i s'eliminen sempre del mateix costat, creant un comportament LIFO.

Operacions principals

push(x)	Afegir element al tope de la pila
pop()	Eliminar element del tope
top()	Accedir al element del tope sense eliminar
empty()	Verificar si la pila està buida
size()	Obtenir el nombre total d'elements



TEMA 2 - APLICACIONS

Casos d'ús de Piles

Aplicacions reals on les piles són la solució natural

Inversió de seqüències

Invertir l'ordre d'una seqüència apilant i desapilant els elements.

```
Entrada: 1,2,3,4 → Push tots; Pop tots → Sortida: 4,3,2,1
```

Avaluació d'expressions

Avaluar expressions en RPN: operands a la pila, operadors els desapilen.

```
"3 4 +" → Push 3, Push 4; operador + → resultat 7
```

Pila d'execució

Gestionar crides a funcions amb adreces de retorn a la pila.

```
f_a crida f_b → Push adreça retorn; al acabar → Pop i retorna
```

Validació de parèntesis

Comprovar que parèntesis i claudàtors estiguin balancejats.

```
"{[()]}" → Vàlid; "{[(])}" → Invàlid
```

Proper element major

Trobar per a cada element el següent element més gran a la dreta.

```
Array: [4,5,2,25] → Resultat: [5,25,25,-1]
```

TEMA 2 - IMPLEMENTACIÓ

Implementació d'una Pila en C++

Ús de templates genèrics, vector i mètodes inline

Declaració de la classe

```
template <typename T>
class Stack {
  std::vector<T> elems_;

public:
  Stack() {}

  Stack(const std::vector<T>&
                  elems)
    : elems_(elems) {}

  int size() const {
    return elems_.size();
  }
};
```

Templates genèrics: La classe pot treballar amb qualsevol tipus de dades (int, double, string, etc.) gràcies a template <typename T>.

Mètodes principals

```
bool empty() const {
  return elems_.empty();
}

void push(const T& x) {
  elems_.push_back(x);
}

const T& top() const {
  assert(elems_.size() > 0,
                 "top: pila buida!");
  return elems_.back();
}

void pop() {
  assert(elems_.size() > 0,
```

Tots els mètodes són inline: S'implementen directament dins la classe, per a eficiència en operacions molt curtes.

TEMA 2 - CUES

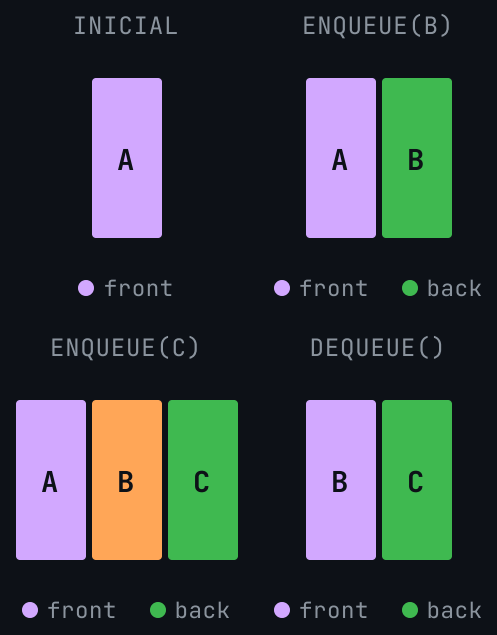
Cues: Definició i Operacions

TAD FIFO (First In First Out) - El primer que entra és el primer que surt

> Una **cua** és un contenidor FIFO on els elements s'afegeixen al final i s'eliminen del principi. Semblant a una cua de persones en un supermercat.

Operacions principals

enqueue(x)	Afegir element al final de la cua
dequeue()	Eliminar element del principi
front()	Accedir al primer element sense eliminar
back()	Accedir a l'últim element afegit
empty()	Verificar si la cua està buida
size()	Obtenir el nombre total d'elements



TEMA 2 - COMPARATIVA

Piles vs Cues

Diferències fonamentals entre dues estructures de dades lineals

Pila (Stack)

ESTRUCTURA LIFO

Ordre: LIFO (Last In First Out)

Accés: push() / pop() al tope

Operacions: O(1) per a totes

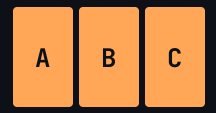
Recorregut: No es pot recórrer

Casos d'ús:

- Inversió de seqüències
- Avaluació RPN
- Pila d'execució
- Validació parèntesis

Cua (Queue)

ESTRUCTURA FIFO



Ordre: FIFO (First In First Out)

Accés: enqueue() / dequeue()

Operacions: O(1) per a totes

Recorregut: No es pot recórrer

Casos d'ús:

- Cues de processament
- Sistemes de cues
- BFS (graph)
- Simulacions

TEMA 2 - IMPLEMENTACIÓ

Implementació d'una Cua en C++

Estructura similar a Pila però amb accés FIFO

Declaració de la classe

```
template <typename T>
class Queue {
  std::deque<T> elems_;

public:
  Queue() {}

  int size() const {
    return elems_.size();
  }

  bool empty() const {
    return elems_.empty();
  }
};
```

Contenidor deque: Permet accés eficient tant al principi com al final, ideal per a cues.

Mètodes principals

```
void enqueue(const T& x) {
  elems_.push_back(x);
}

const T& front() const {
  assert(elems_.size() > 0,
                 "front: cua buida!");
  return elems_.front();
}

void dequeue() {
  assert(elems_.size() > 0,
                 "dequeue: cua buida!");
  elems_.pop_front();
}
```

Diferència clau: Mentre que Stack usa push_back/pop_back, Queue usa push_back/pop_front per mantenir FIFO.

TEMA 2 - EXEMPLE PRÀCTIC

Avaluació d'expressions en RPN

Notació Polonesa Inversa (Reverse Polish Notation) amb pila

Algoritme

```
int evalua_RPN(string expr) {
  Stack<int> S;
  for (each token in expr) {
    if (isNumber(token)) {
      S.push(token);
    } else {
      int b = S.pop();
      int a = S.pop();
      S.push(opera(a, b, token));
    }
  }
  return S.top();
}
```

Exemple: "3 4 + 2 *"

Expressió: "3 4 + 2 *"
Equivalent infixa: (3 + 4) * 2 = 14

Pas 1: Token "3"
Pila: [3]

Pas 2: Token "4"
Pila: [3, 4]

Pas 3: Token "+"
Pop 4, Pop 3 → 3+4=7
Pila: [7]

Pas 4: Token "2"
Pila: [7, 2]

Pas 5: Token "*"
Pop 2, Pop 7 → 7*2=14
Pila: [14] ✓

TEMA 2 - APLICACIONS

Casos d'ús de Cues

Aplicacions reals on les cues són essencials

Cues de processament

Sistemes que processen tasques en ordre FIFO. La primera tasca que arriba és la primera que es processa.

Exemple: Impressora de xarxa, cua de descàrregues

Sistemes de cues

Gestió de clients en espera. Bancs, botigues, hospitals utilitzen cues per ordre d'arribada.

Exemple: Sistema de tickets, cua de pacients

BFS (Breadth-First Search)

Algoritme de recorregut en grafs. Explora nodes per nivells, usant cua per mantenir l'ordre.

Exemple: Camí més curt, exploració de xarxa

Simulacions

Modelar sistemes reals com tràfic, cues de supermercat, sistemes de telecomunicacions.

Exemple: Simulació de tràfic, cua de clients

CPU Scheduling

Sistemes operatius usen cues per gestionar processos en espera de temps de CPU.

Exemple: Round-robin scheduling, cua de processos

TEMA 2 - RESUM

Conceptes clau de Piles i Cues

Resum dels aprenentatges principals del tema

Piles (Stack)

Estructura LIFO on els elements s'afegeixen i s'eliminen del tope. Operacions: push, pop, top, empty, size. Tots O(1).

Cues (Queue)

Estructura FIFO on els elements s'afegeixen al final i s'eliminen del principi. Operacions: enqueue, dequeue, front, empty, size. Tots O(1).

Implementació

Ambdues usen vector/deque com a contenidor intern. Mètodes inline per eficiència. Templates genèrics per a qualsevol tipus de dada.

Validació

Ús de assert per validar pre-condicions. Convencions de noms (membre_, _membre, m_membre) per a membres privats.

Punts clau a recordar

- ✓ Piles i cues són estructures lineals amb accés restringit
- ✓ La diferència fonamental és l'ordre d'accés: LIFO vs FIFO
- ✓ Ambdues ofereixen operacions O(1) per a totes les operacions
- ✓ Aplicacions reals: RPN, cues de processament, BFS, simulacions
- ✓ Templates permeten reutilitzar el codi per a qualsevol tipus de dada
- ✓ Mètodes inline optimitzen el rendiment per a operacions curtes

// Tema 2 completat
// Piles i Cues
// Estructures fonamentals

Felicitats!
Has dominat Piles i Cues

Ara comprens dues estructures de dades fonamentals amb accés restringit. Aquestes estructures són la base per a molts algoritmes i sistemes reals.

→ Pròxim: Tema 3 - Llistes

En el Tema 3 explorarem les **llistes enlla\u00e7ades**, una alternativa més flexible als vectors. Aprendrem com implementar piles i cues usant llistes, i com treballar amb estructures dinàmiques que creixen i decreixen durant l'execució del programa.

Continua amb el Tema 3 →

// Tema 3: Llistes
// Estructures dinàmiques
// Pròxim: Llistes
enlla\u00e7ades